

Running FlowForge locally in VS Code (npm)

This repo is an **npm workspaces** monorepo. Don't run `npm i -f`; just `npm install` from the repo root.

1. Prerequisites

- Node.js 20+ and npm 10+ (check with `node -v` / `npm -v`)
- Docker Desktop, for Postgres + Redis only — the apps themselves run directly on your machine (not in Docker) so VS Code's debugger/terminal can attach to them.

2. Open the repo in VS Code

```
code flowforge
```

Open one integrated terminal per step below (`Ctrl+`` / `Cmd+``, then the `+` icon to split) — you'll want up to 4 terminals running at once in steps 6-8.

3. Install dependencies (root only — installs every workspace)

```
npm install
```

Run this from the **repo root**, not from `apps/api` or `apps/web` individually — npm workspaces need the root install to link `@flowforge/shared-types` and `@flowforge/node-sdk` into the apps that depend on them.

4. Environment variables

```
cp .env.example .env
```

The defaults point at `localhost:5432` (Postgres) and `localhost:6379` (Redis), which is exactly what `docker-compose.yml` exposes to the host in step 5 below.

5. Start Postgres + Redis in Docker

```
docker compose up -d postgres redis
```

This only starts the `postgres` and `redis` services — not `api/worker/web`, which you'll run natively via npm instead so VS Code can see them directly. Confirm both are

healthy:

```
docker compose ps
```

6. Apply the database schema (includes the Phase 10 log-streaming migration)

```
npm run prisma:migrate:deploy
```

This runs `prisma migrate deploy` inside `apps/api`, applying every migration under `apps/api/prisma/migrations/*` in order — including `00000000000010_log_streams_presence`, which adds the `LogStreamConfig` table. Equivalent manual form if you ever need it:

```
cd apps/api && npx prisma migrate deploy && cd ../..
```

7. Build the shared packages once

```
npm run build:shared-types  
npm run build:node-sdk
```

8. Run the app (3 separate terminals, from the repo root)

```
npm run dev:api  
npm run dev:worker  
npm run dev:web
```

Each maps to a workspace script (`npm run dev --workspace=@flowforge/api`, etc.), so you can also run them individually with `--workspace=@flowforge/api / @flowforge/worker / @flowforge/web` if you'd rather not use the root aliases.

Open `http://localhost:5173` once `dev:web` is up.

9. (Optional) Browser-automation sidecar

Only needed for the `browserAutomation` node:

```
cd services/browser-runner  
npm install
```

```
npm start
```

10. Shutting down

```
# Stop the npm dev processes with Ctrl+C in each terminal, then:
docker compose down          # stop Postgres/Redis, keep data
docker compose down -v       # stop and wipe the Postgres/Redis volumes
```

Quick reference — every command in one place

What	Command
Install all workspace deps	<code>npm install</code>
Start Postgres + Redis	<code>docker compose up -d postgres redis</code>
Apply DB migrations	<code>npm run prisma:migrate:deploy</code>
Build shared packages	<code>npm run build:shared-types && npm run build:node-sdk</code>
Run API	<code>npm run dev:api</code>
Run worker	<code>npm run dev:worker</code>
Run web (Vite dev server)	<code>npm run dev:web</code>
Build everything for prod	<code>npm run build</code>
New Prisma migration (dev)	<code>cd apps/api && npx prisma migrate dev --name <migration_name></code>
Open Prisma Studio	<code>cd apps/api && npx prisma studio</code>
Stop Docker services	<code>docker compose down</code>

What changed from the original pnpm setup

- Root `package.json` now has a `"workspaces": ["apps/*", "packages/*"]` field instead of `pnpm-workspace.yaml` (removed).
- Every internal `"workspace:*` dependency (e.g. `@flowforge/shared-types`, `@flowforge/node-sdk`) was changed to `"*"`, which is what npm workspaces expects for symlinking local packages.
- Root scripts (`dev:api`, `dev:worker`, `dev:web`, `build`, `lint`) now use `npm run <script> --workspace=<name> / --workspaces` instead of pnpm's `--filter`.
- `pnpm-lock.yaml` was removed; a fresh `package-lock.json` will be generated the first time you run `npm install`.

- `docker-compose.yml`'s `postgres/redis` services now publish 5432/6379 to the host (they didn't before), since apps run natively via npm in this workflow rather than as Docker services themselves.
- Fixed several TypeScript errors uncovered by a full `tsc --noEmit` pass across every package (API, worker, web, and the two shared packages) — see the "Syntax/type fixes" section below.

Syntax/type fixes made

- `apps/api/src/routes/workflowVersions.ts` — the rollback route called the unsupported `router.handle()` method; publish logic was extracted into a shared `publishVersion()` function used by both the publish and rollback routes instead.
- `apps/api/src/utils/emailPoller.ts` — added `@types/mailparser` and typed an implicit-any callback parameter.
- `apps/api/src/utils/saml.ts` — `@node-saml/node-saml`'s config field is `cert`, not `idpCert`; also fixed a `Record<string, unknown>` fallback typing.
- `apps/api/src/utils/triggerPollers.ts` — cast the Kafka `sasl` option to `SASLOptions` (kafkajs's discriminated union couldn't be inferred from our looser config shape), and reordered `xreadgroup`'s `COUNT/BLOCK` args to match `ioredis`'s typed overload.
- `apps/worker/src/engine/executor.ts` — narrowed `subgraph` to non-undefined after its guard clause, and corrected a `Map<string, unknown>` that should have been `Map<string, NodeItems>` when restoring a paused execution's checkpoint.
- `apps/worker/src/engine/expressions.ts` — `random.int(min, max)` helper wasn't accepting the `args` it was called with; gave it an optional `args` parameter and real range support.
- `apps/worker/src/index.ts` / `apps/worker/src/db/executions.ts` — widened the trigger-type union to include `emailTrigger` / `fileWatcher` / `databaseChange` / `streamTrigger`, matching what `@flowforge/shared-types` already declared.
- `apps/worker/src/nodes/agentNode.ts` — cast two `unknown` stage outputs before spreading them into the reasoning trace.
- `apps/worker/src/nodes/ragNode.ts` — `resolveDocuments`'s `items` param now uses the shared `NodeItems` type instead of a stricter local shape that didn't match real node item data (where `binary.data` can be undefined).

All packages (`apps/api`, `apps/worker`, `apps/web`, `packages/shared-types`, `packages/node-sdk`, `packages/node-cli`) now pass `tsc --noEmit` cleanly, and `apps/web` builds successfully with `vite build`.